Open Problems in Formal Verification of Modular Compositional AI Systems

Kim Daniel Schulte-Zurhausen[1]

[1]Modulith Research CIC, London, UK, research@modulith.ai

March 2026

Abstract

We describe an AI architecture in which independently trained expert modules are permanently frozen after creation and dynamically composed through sparse top-K routing in a shared representation space. This architecture differs structurally from standard mixture-of-experts systems in four respects: modules are trained independently (no shared gradients), permanently frozen (immutable functions), dynamically added or removed (open pool), and composed via an explicit sparse graph at each inference step. We present empirical evidence from 260+ experiments suggesting that these structural constraints may enable formal mathematical reasoning about safety-relevant properties, specifically attribution, guaranteed forgetting, non-degradation, memory isolation, and compositional monotonicity, that are intractable in jointly-trained architectures. We state each property precisely and pose open mathematical questions amenable to formal analysis, inviting contributions from the formal methods, learning theory, and information theory communities.

Repository: https://github.com/kdschulte/storm-safety-proofs

1 Introduction

The safety verification of neural networks is a central challenge in AI safety research [1]. For monolithic architectures, including transformers, large language models, and jointly-trained mixture-of-experts systems [12, 6], formal reasoning about properties such as knowledge attribution [13], guaranteed forgetting [3], and compositional stability remains intractable in practice. The fundamental difficulty is that knowledge in these systems is distributed across entangled parameters [4], making it impossible to localise influence, bound contributions, or guarantee removal of specific knowledge.

We study an architecture that introduces structural constraints designed to make such reasoning tractable. The system consists of a pool of independently trained expert modules, each frozen after creation, composed at inference time through sparse top-K routing [12] in a shared representation space. These constraints—independent training, permanent freezing, dynamic pool membership, and explicit composition graphs—collectively produce a class of compositional AI system that is structurally different from standard mixture-of-experts.

This paper makes two contributions:

1. We state six safety-relevant properties precisely, describe the structural constraints that may make them provable, and present empirical evidence from 260+ experiments motivating formal treatment.
2. We pose each property as an open mathematical question, identifying the relevant proof strategies and inviting contributions from the formal methods, learning theory, and information theory communities.

This is not a theoretical paper claiming proofs. It is a research programme statement: here are the questions, here is why we believe they are tractable, and here is the evidence that the structural constraints produce the expected mathematical behaviour in practice.

2 Architecture

Definition 1 (Modular Compositional System)**.** *A modular compositional system $\mathcal{S}$ consists of:*

- *A **module pool** $\mathcal{M} = \{m_1, m_2, \ldots, m_N\}$ where each $m_i : \mathbb{R}^d \to \mathbb{R}^d$ is a frozen function mapping from and to a shared representation space of dimension d.*
- *A **router** R that, for each input, selects a subset $S(\mathbf{x}) \subset \{1, \ldots, N\}$ with $|S| = K \ll N$ and assigns non-negative weights $\{w_n(\mathbf{x})\}_{n \in S}$ with $\sum_{n \in S} w_n = 1$.*
- *A **shared backbone** providing the representation space and any shared computation (e.g., attention, embedding).*

For input representation $\mathbf{x} \in \mathbb{R}^d$ at position t, let $S_t(\mathbf{x}) \subset \{1, \ldots, N\}$ with $|S_t| = K$ be the subset selected by R, and let $w_n(\mathbf{x}) \geq 0$ for $n \in S_t$ with $\sum_{n \in S_t} w_n = 1$. The composed output is:

$$\mathbf{y}_t = \sum_{n \in S_t(\mathbf{x})} w_n(\mathbf{x}) \cdot m_n(\mathbf{x}), \quad |S_t| = K \ll N \tag{1}$$

This formalises the modular architecture pattern described in the recent survey by Pfeiffer et al. [11], with the additional constraints of independent training and permanent freezing.

2.1 Structural Constraints

Four constraints distinguish this architecture from standard mixture-of-experts:

1. **Independent training.** Each module m_i is trained on its own data using its own loss function. No gradients flow between modules during training. Modules have no knowledge of each other's existence.
2. **Permanent freezing.** Once trained, module weights are immutable: $\nabla_{\theta_i} = 0$ for all i during and after composition. Each module is a fixed function.
3. **Dynamic pool membership.** Modules can be added to or removed from $\mathcal{M}$ at any time. Adding module m_{N+1} requires no retraining of $m_1, \ldots, m_N$. Removing module m_j requires only updating the router.
4. **Explicit composition graph.** Each inference step over a sequence of length T induces a sparse bipartite graph $G = (V_{\text{pos}}, V_{\text{mod}}, E)$ where edge $(t, i) \in E$ has weight $w_{t,i}$ recording the contribution of module m_i to position t. This graph is a concrete, observable mathematical object.

Remark 1. *In standard MoE [12] (Switch Transformer [6], GShard [8], Mixtral [9]), experts share gradients during training, weights are mutable, the pool is fixed at training time, and knowledge is distributed across all experts. These design choices make the same formal questions intractable for the same reasons they are intractable in monolithic models.*

The distinction is not merely architectural but has direct consequences for formal reasoning. Because modules are independently trained, no module's parameters encode information about another module's training data—this is the structural basis for guaranteed forgetting. Because modules are permanently frozen, the composition equation (Equation 1) is a convex combination of fixed, independent functions—this is what makes attribution meaningful rather than nominal. Because the pool is dynamic, modules can be added or removed without retraining existing modules—this is what makes non-degradation and compositional monotonicity tractable questions. None of these formal properties can be stated, let alone proven, for jointly trained MoE systems where expert representations are entangled through shared gradient updates.

3 Safety Properties and Open Problems

We state six safety-relevant properties, describe the empirical evidence motivating each, and pose the associated open mathematical questions.

3.1 Attribution

Property 1 (Attribution). *For any output token y_t produced by the system, there exists a decomposition of the output into per-module contributions with explicit weights, such that the contribution of each module m_i is identifiable and bounded.*

Empirical evidence. The composition rule (Equation 1) provides attribution directly: the weight w_n assigned to each selected module m_n at position t quantifies its contribution. In our experiments with modules across multiple knowledge domains, lightweight domain-specific modules achieve discrimination ratios exceeding two orders of magnitude, earning high routing weight on their domain and near-zero weight on unrelated inputs.

Conjecture 1 (Meaningful Attribution). *Under what conditions on the router R and modules $\{m_i\}$ does the weight decomposition in Equation 1 constitute a* meaningful *attribution, i.e., one where removing a module with weight w from the composition changes the output by $\Theta(w)$?*

Proof strategy. The convex combination structure provides a natural starting point. The key question is whether the modules are sufficiently "spread" in the representation space that weight directly corresponds to influence. This may connect to the theory of convex decompositions and basis pursuit. Note that standard MoE systems also have gating weights, but those weights do not constitute meaningful attribution because jointly trained experts develop correlated, entangled representations—removing an expert changes the behaviour of the remaining experts through shared gradient history. In the frozen-module architecture, each module is an independent fixed function, so the convex decomposition is over genuinely independent terms.

3.2 Guaranteed Forgetting

Property 2 (Guaranteed Forgetting). *Removing module m_j from the pool $\mathcal{M}$ eliminates all knowledge associated with m_j from all future outputs of the system.*

Empirical evidence. In our experiments, removing a domain-specific module eliminates that domain's content from outputs. A substantial fraction of routing weight is allocated to memory-type modules; removing a memory module causes the router to redistribute weight among remaining modules with no trace of the removed content.

Conjecture 2 (Information-Theoretic Forgetting). *Let D_j be the training data of module m_j. After removing m_j from $\mathcal{M}$ and retraining the router on the reduced pool, is the mutual information $I(D_j; Y_t) = 0$ for all future outputs Y_t? Under what conditions on the shared backbone and remaining modules does this hold?*

Proof strategy. The permanent freezing constraint is key: since no other module's weights were influenced by D_j during training (independent training), and no other module's weights changed after m_j was added (frozen), any information about D_j in the system is localised to m_j itself and the router's learned preferences. This is a *structural guarantee* that standard MoE cannot provide: in jointly trained systems, knowledge from any training example may be distributed across all experts and the shared backbone, making complete removal intractable [3]. The open questions are: (1) the router's residual information about D_j after retraining, and (2) whether the shared backbone (which is not modular) leaks information about D_j if the backbone was exposed to related data.

3.3 Non-Degradation

Property 3 (Non-Degradation)**.** *Adding a new module m_{N+1} to the pool does not degrade the system's performance on any existing capability.*

Empirical evidence. Expanding the module pool by approximately 25% preserves evaluation quality within tight bounds. Leave-one-out ablation confirms all domain groups contribute positively.

Conjecture 3 (PAC-Style Non-Degradation)**.** *Given a pool $\mathcal{M}$ of N frozen modules with router R achieving expected loss L_N on distribution $\mathcal{D}$, adding module m_{N+1} and retraining only R yields expected loss $L_{N+1} \leq L_N + \epsilon$ with probability at least $1 - \delta$, for some ϵ, δ that depend on the number of routing samples and the capacity of R.*

Proof strategy. Since existing modules are frozen and only the router changes, this becomes a question about the sample complexity of learning a routing policy over $N + 1$ options versus N options. PAC-Bayes bounds [14] or Rademacher complexity arguments for the router class may apply.

3.4 Memory Isolation

Property 4 (Memory Isolation)**.** *For $K \ll N$, a module m_j designated as user-specific has negligible influence on outputs for other users. Specifically, for inputs $\mathbf{x}$ not associated with user j, the routing weight $w_j(\mathbf{x}) \approx 0$.*

Empirical evidence. The sparse top-K routing mechanism means that at most K modules contribute to any given output. When $K \ll N$, a user-specific module is selected only when the router deems it relevant. In our experiments with $K \ll N$, domain-specific modules receive near-zero routing weight on unrelated inputs, with discrimination ratios exceeding two orders of magnitude.

Conjecture 4 (Differential Privacy of Routing)**.** *Can the sparse routing mechanism be shown to satisfy (ϵ, δ)-differential privacy [5] with respect to the presence or absence of a specific module in the pool? If so, what are the privacy parameters as a function of K, N, and the router architecture?*

Proof strategy. The sparsity of routing ($K \ll N$) limits the influence of any single module. This may connect to the literature on differential privacy in mechanism design [5], where the "mechanism" is the router and the "database" is the module pool.

3.5 Bounded Compute

Property 5 (Bounded Compute)**.** *Module execution cost is $O(K)$ per position, where K is the number of selected modules. Router selection cost is $O(N)$.*

This property follows directly from the architecture: the router evaluates affinities for all N modules in $O(N)$ (or selects the top K via a heap in $O(N \log K)$), while the composition step invokes exactly K module forward passes. Total inference cost per position is therefore $O(N + K \cdot C_{\text{mod}})$, where C_{mod} is the cost of a single module forward pass. When $C_{\text{mod}} \gg N$ (typical for transformer-based modules), total cost is dominated by the K module executions and is effectively constant in N. However, for very large pools, the $O(N)$ router cost may become non-negligible, motivating hierarchical or approximate routing strategies. This is the only property we consider proven from the architecture definition.

3.6 Compositional Monotonicity

Property 6 (Compositional Monotonicity)**.** *Adding a domain-specific module m_j trained on domain $\mathcal{D}_j$ improves performance on $\mathcal{D}_j$ without degrading performance on other domains $\mathcal{D}_i$ for $i \neq j$.*

Empirical evidence. Leave-one-out ablation across all domain groups confirms that each group contributes positively to its own evaluation set and does not degrade others.

Conjecture 5 (Finite-Sample Monotonicity)**.** *Under what conditions on the module training procedure and router architecture does adding a module trained on $\mathcal{D}_j$ improve expected loss on $\mathcal{D}_j$ by at least $\gamma > 0$ while increasing expected loss on $\mathcal{D}_i$ $(i \neq j)$ by at most ϵ, with high probability?*

Proof strategy. This combines the non-degradation argument (router sample complexity) with a domain-specific improvement bound. The key insight is that if m_j is the best available module for inputs from $\mathcal{D}_j$, the router will learn to select it, improving domain performance. The non-degradation bound constrains the cost on other domains.

Remark 2. *Monotonicity as stated is pairwise: adding one module improves its domain without degrading others. This does not immediately imply transitivity—adding modules m_a and m_b individually may each satisfy the bound, but adding both may not, since the router must now allocate K selections among a larger pool. A stronger compositional monotonicity result would require bounds on the interaction between simultaneously added modules.*

4 Experimental Summary

The architecture has been implemented and tested across 260+ experiments on consumer hardware. Key results that motivate the formal programme are summarised in Table 1.

Table 1: Summary of experimental observations motivating formal treatment.

Observation	**Evidence**	**Motivates**
Modules compose to outperform individuals	Routed pool outperforms best single module	Attribution, Mor
Sharp domain discrimination	Ratios exceeding two orders of magnitude	Attribution, Isola
Pool expansion preserves quality	Quality stable under ∼25% pool growth	Non-Degradation
Learned router utilises most modules	Majority of pool selected across inputs	Composition Gra
Memory modules carry substantial weight	Removable, localisable knowledge	Forgetting, Isolat
Leave-one-out: all domains positive	Every domain group contributes positively	Monotonicity

Full experimental data and sanitised datasets are available in the public repository.

5 Related Work

Mixture-of-Experts. The sparsely-gated MoE layer [12] and its successors Switch Transformer [6], GShard [8], and Mixtral [9] train experts jointly with shared gradients, producing entangled knowledge representations. Our independent training and permanent freezing constraints create a structurally different system.

Modular Networks. Neural module networks [2] compose specialised modules for visual question answering, but modules are jointly trained and task-specific. The broader landscape of modular deep learning is surveyed by Pfeiffer et al. [11]. Our modules are domain-general frozen functions that compose in a shared representation space.

Retrieval-Augmented Generation. RAG systems [10] compose a generator with retrieved documents, but the generator is monolithic and the "composition" is concatenation in input space, not convex combination in representation space.

Machine Unlearning. The growing literature on machine unlearning [3] seeks to remove data influence from trained models. Our architecture may offer a structural solution: if knowledge is localised in a frozen module, "unlearning" is module removal.

Formal Verification of Neural Networks. Existing work on neural network verification [7] focuses on input-output properties of fixed networks (e.g., robustness bounds). Our questions concern compositional properties of modular systems, a different class of formal problem.

6 Discussion

We have described an architecture whose structural constraints—independent training, permanent freezing, dynamic pool membership, and explicit composition graphs—may enable formal mathematical reasoning about safety-relevant properties. We do not claim that these properties are proven. We claim that the structural constraints create mathematical objects (frozen functions, composition graphs, additive knowledge pools) that are amenable to analysis in ways that monolithic and jointly-trained architectures are not.

The empirical evidence from 260+ experiments is encouraging but not sufficient. The research programme requires formal methods expertise, specifically in PAC learning theory [14], information theory, differential privacy [5], and formal verification [7], to translate empirical observations into rigorous results. We invite contributions from these communities.

Stated Assumptions. The following assumptions underlie the properties in Section 3 and are made explicit here:

1. **Frozen backbone at inference.** The shared backbone (attention, embedding layers) is fixed during module composition. Backbone parameters do not change when modules are added, removed, or composed.
2. **Non-overlapping training data is NOT required.** Modules may be trained on overlapping data distributions. The independence guarantee comes from separate training processes (no shared gradients), not from data partitioning.
3. **Router retraining on pool changes.** When a module is added to or removed from $\mathcal{M}$, the router R is retrained (or fine-tuned) on the updated pool. Existing module weights remain frozen during router retraining.
4. **Additive composition in shared space.** The composition rule (Equation 1) is a convex combination in $\mathbb{R}^d$. The properties as stated assume this additive structure; alternative composition rules (e.g., sequential, multiplicative) would require separate analysis.
5. **K is fixed and $K \ll N$.** The memory isolation and bounded compute properties assume K is a fixed constant much smaller than N. At $K = N$, all modules contribute and isolation guarantees degrade.
6. **No minimum N is established.** The empirical evidence is from pools of dozens of modules. Whether the properties hold at $N = 2$ or require a minimum pool size is an open question.

Limitations. The current experimental evidence is on a system with dozens of lightweight modules running on consumer hardware. It is an open question whether the structural properties scale to larger module counts and parameter sizes. The router is a learned component whose theoretical properties require separate analysis. The shared backbone (attention, embedding) is not modular and represents a potential source of entanglement that must be addressed in any complete formal treatment.

Reproducibility. The mathematical specification, sanitised experimental data, and proof agenda are available at https://github.com/kdschulte/storm-safety-proofs.

References

[1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.

[2] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein. Neural module networks. In *CVPR*, 2016.

[3] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot. Machine unlearning. In *IEEE S&P*, 2021.

[4] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Anthropic Research*, 2023.

[5] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[6] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23(120):1–39, 2022.

[7] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *CAV*, 2017.

[8] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *ICLR*, 2021.

[9] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, et al. Mixtral of experts. *arXiv:2401.04088*, 2024.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[11] J. Pfeiffer, S. Ruder, I. Vulić, and E. Ponti. Modular deep learning. *TMLR*, 2023.

[12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

[13] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. In *ICML*, 2017.

[14] L. G. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.